

Encapsulamento

Proteção de dados e organizando métodos

Linguagem de Programação II

Contexto: conta bancária

Vamos começar criando uma classe para representar uma conta bancária simples.

Problema

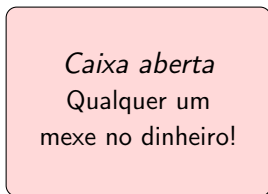
Uma conta bancária precisa guardar o nome do **titular** e o **saldo** disponível.

O que a conta deve permitir?

- Criar uma conta para um titular.
- Começar com saldo igual a zero.
- Permitir depósitos.
- Permitir saques apenas quando houver saldo suficiente.
- Consultar o titular e o saldo da conta.

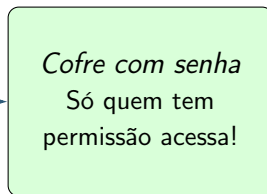
Antes de começar: uma analogia

Sem Encapsulamento



proteger

Com Encapsulamento



Ideia central

Encapsulamento é como colocar seus dados num cofre: apenas métodos autorizados podem acessá-los ou modificá-los.

Problema inicial: conta bancária sem proteção

Imagine uma conta bancária representada assim em Java:

```
class ContaBancaria {  
    String titular;  
    double saldo;  
}
```

No main, qualquer código pode fazer isso:

```
ContaBancaria conta = new ContaBancaria();  
conta.titular = "Maria";  
conta.saldo = -5000; // Isso nao deveria ser permitido!
```



Problema

Uma conta bancária com saldo negativo logo ao ser criada? Sem nenhuma regra? Isso é um sério problema de segurança!

O que pode dar errado com atributos públicos?

✗ Problemas

- Saldo negativo sem motivo
- Nota maior que 10
- Preço de produto negativo
- Idade negativa
- Nomes em branco

✓ Com encapsulamento

- Saldo só muda via `depositar()` e `sacar()`
- Nota validada entre 0 e 10
- Preço só aceita valores positivos
- Idade não pode ser negativa
- Nome não pode ser vazio

Resumo

Sem encapsulamento, o objeto **perde o controle** sobre seus próprios dados. A classe precisa ser a **guardiã** das suas informações.

Visualizando o problema



- O objeto **perde o controle** sobre seus próprios dados.
- Valores inválidos podem ser inseridos **sem nenhuma verificação**.
- A regra de negócio fica **espalhada** pelo sistema.
- Quando der erro, **onde buscar?** Em vários lugares diferentes!

Modificadores de acesso em Java

Em Java, controlamos quem pode acessar o quê usando **modificadores de acesso**:

Modificador	O que significa na prática
<code>public</code>	Todo mundo pode usar
<code>protected</code>	Só a família (subclasses) e vizinhos (mesmo pacote)
<i>default</i>	Só os vizinhos (mesmo pacote)
<code>private</code>	Só eu mesmo (dentro da classe)

💡 Regra de ouro para iniciantes

- **Atributos** → sempre `private`
- **Métodos que outros precisam usar** → `public`

Aplicando encapsulamento: passo 1

Transformando os atributos em private:

```
public class ContaBancaria {  
  
    private String titular;    // So acessivel dentro desta classe  
    private double saldo;     // So acessivel dentro desta classe  
  
    public ContaBancaria(String titular) {  
        this.titular = titular;  
        this.saldo = 0; // Saldo sempre começa em zero!  
    }  
  
    public double getSaldo() { // Metodo publico para consultar  
        return saldo;  
    }  
  
    public String getTitular() { // Metodo publico para consultar  
        return titular;  
    }  
}
```

- Saldo e titular **não** podem mais ser alterados diretamente.
- O acesso acontece por meio de **métodos** controlados.

O que acontece com acesso direto agora?

Com os atributos privados, este código **gera erro de compilação**:

```
ContaBancaria conta = new ContaBancaria("Maria");  
  
conta.saldo = 1000; // ERRO: saldo tem acesso private  
System.out.println(conta.saldo); // ERRO: saldo tem acesso private
```

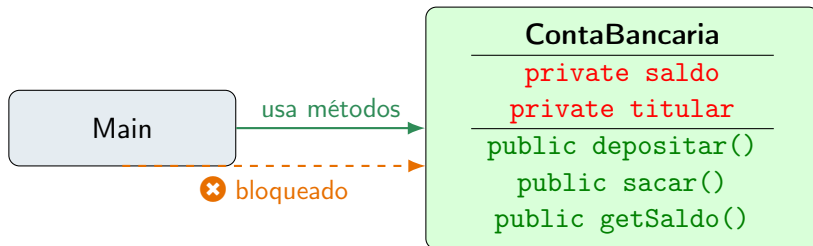
 Mensagem de erro no compilador

saldo has private access in ContaBancaria

 A forma correta

```
System.out.println(conta.getSaldo()); // Correto!
```

Visualizando a solução



O que mudou?

O Main não acessa mais os dados diretamente. Ele **pede à classe** que realize as operações necessárias.

Método depositar: a regra fica na classe

```
public void depositar(double valor) {  
    if (valor > 0) {  
        saldo = saldo + valor;  
        System.out.println("Deposito de R$ " + valor + " realizado."  
            );  
    } else {  
        System.out.println("Valor invalido para deposito.");  
    }  
}
```

✓ Permitido

- depositar(100.0) → OK
- depositar(500.50) → OK

✗ Bloqueado

- depositar(-50.0) → Negado
- depositar(0) → Negado

Por que isso importa?

A regra "depósito deve ser positivo" fica dentro da classe, não espalhada pelo código.

Método sacar: verificando saldo disponível

```
public boolean sacar(double valor) {  
    if (valor > 0 && valor <= saldo) {  
        saldo = saldo - valor;  
        return true;    // Saque bem-sucedido  
    }  
    return false;      // Saque negado  
}
```

✓ Retorna true

Saldo = R\$1000

sacar(300) → OK ✓

Saldo vira R\$700

✗ Retorna false

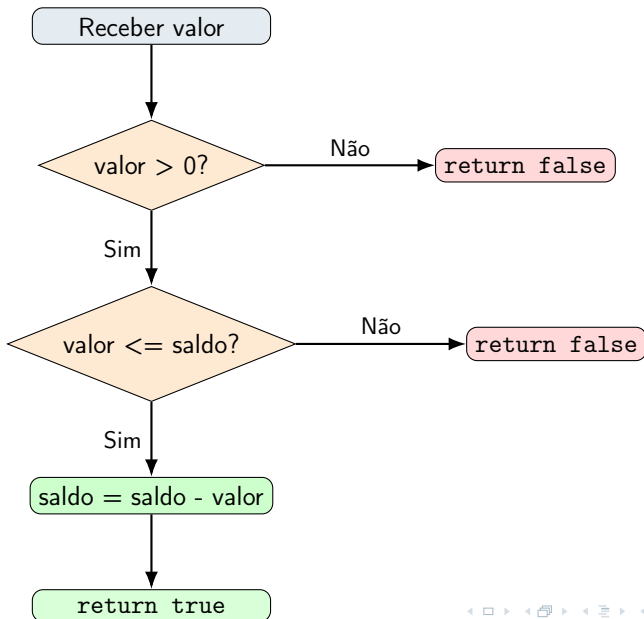
Saldo = R\$1000

sacar(1500) → Negado

Saldo continua R\$1000

- O método retorna boolean → true se deu certo, false se não.
- O main pode usar essa resposta para informar o usuário.

Fluxograma do método sacar()



Classe ContaBancaria completa

```
public class ContaBancaria {  
  
    private String titular; // atributo privado  
    private double saldo;   // atributo privado  
  
    // Construtor: garante saldo inicial = 0  
    public ContaBancaria(String titular) {  
        this.titular = titular;  
        this.saldo = 0;  
    }  
  
    // Operacao de deposito (com validacao)  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo = saldo + valor;  
        }  
    }  
  
    // Operacao de saque (com validacao, retorna sucesso ou falha)  
    public boolean sacar(double valor) {  
        if (valor > 0 && valor <= saldo) {  
            saldo = saldo - valor;  
            return true;  
        }  
        return false;  
    }  
  
    // Getters: permitem apenas leitura  
    public double getSaldo() { return saldo; }  
    public String getTitular() { return titular; }  
}
```

Usando a classe no Main

```
class Main {
    public static void main(String[] args) {

        ContaBancaria conta = new ContaBancaria("Maria");

        conta.depositar(1000);           // Deposita R$ 1000
        boolean ok = conta.sacar(300);  // Tenta sacar R$ 300

        System.out.println("Titular: " + conta.getTitular());
        System.out.println("Saldo: R$ " + conta.getSaldo());

        if (ok) {
            System.out.println("Saque realizado com sucesso!");
        } else {
            System.out.println("Saque negado: saldo insuficiente.");
        }
    }
}
```

> Saída esperada

Titular: Maria

Getter e Setter: o que são?

Getter (obter)

Método que **retorna** o valor de um atributo privado.

Convenção de nome:

get + NomeDoAtributo

Exemplos:

```
getSaldo()
```

```
getTitular()
```

```
getNome()
```

Setter (definir)

Método que **altera** o valor de um atributo privado (com validação!).

Convenção de nome:

set + NomeDoAtributo

Exemplos:

```
setPreco(double p)
```

```
setNome(String n)
```

```
setIdade(int i)
```

Atenção

Criar setter **sem validação** é quase o mesmo que deixar o atributo público!
Sempre valide.

Exemplo: classe Produto com getter/setter

```
class Produto {  
  
    private String nome;  
    private double preco;  
  
    // Setter com validacao: nome nao pode ser nulo ou vazio  
    public void setNome(String nome) {  
        if (nome != null && !nome.isBlank()) {  
            this.nome = nome;  
        }  
    }  
  
    // Setter com validacao: preco nao pode ser negativo  
    public void setPreco(double preco) {  
        if (preco >= 0) {  
            this.preco = preco;  
        }  
    }  
  
    public double getPreco() { return preco; }  
    public String getNome() { return nome; }  
}
```

Setter com vs. sem validação

✗ Setter fraco (inútil)

```
1 public void setPreco(double preco) {  
2     // Sem validacao!  
3     this.preco = preco;  
4 }  
5  
6 // Qualquer coisa passa:  
7 p.setPreco(-999); // OK :(  
8
```

✓ Setter forte (correto)

```
1 public void setPreco(double preco) {  
2     if (preco >= 0) { // Valida!  
3         this.preco = preco;  
4     }  
5 }  
6  
7 // Valor invalido ignorado:  
8 p.setPreco(-999); // Ignorado :)
```

💡 Lembrete

O setter é a **porta de entrada** do atributo. Se a porta não tem segurança, de nada adianta o cofre.

Nem todo atributo precisa de setter!

Atributo	Getter?	Setter?
saldo da conta	✓ Sim	✗ Não — use depositar/sacar
titular da conta	✓ Sim	? Depende da regra
preço do produto	✓ Sim	✓ Sim (com validação)
nota do aluno	✓ Sim	✓ Sim (0 a 10)

⚠ Armadilha comum

Muitos iniciantes criam getter e setter para **todos** os atributos automaticamente. Isso pode **enfraquecer o encapsulamento**. Pense: *"faz sentido alterar esse atributo diretamente?"*

Exemplo ruim: classe Aluno sem encapsulamento

```
public class Aluno {  
    String nome;        // acesso default (nao encapsulado)  
    double nota1;       // acesso default (nao encapsulado)  
    double nota2;       // acesso default (nao encapsulado)  
  
    Aluno(String nome, double nota1, double nota2) {  
        this.nome = nome;  
        this.nota1 = nota1;  
        this.nota2 = nota2;  
    }  
}
```

✗ Problema

Atributos e construtor com acesso **default** (sem modificador). Classes no mesmo pacote podem fazer:

```
aluno.nota1 = 15;    // Nota impossivel!  
aluno.nota2 = -3;    // Nota negativa!
```

Classe Aluno encapsulada corretamente

```
public class Aluno {  
  
    private String nome;  
    private double nota1;  
    private double nota2;  
  
    public Aluno(String nome, double nota1, double nota2) {  
        setNome(nome);    // Usa o setter para validar ja no construtor!  
        setNota1(nota1);  
        setNota2(nota2);  
    }  
  
    public void setNome(String nome) {  
        if (nome != null && !nome.isBlank()) {  
            this.nome = nome;  
        }  
    }  
  
    public void setNota1(double nota1) {  
        if (nota1 >= 0 && nota1 <= 10) { // So aceita entre 0 e 10  
            this.nota1 = nota1;  
        }  
    }  
  
    public void setNota2(double nota2) {  
        if (nota2 >= 0 && nota2 <= 10) {  
            this.nota2 = nota2;  
        }  
    }  
}
```

A classe calcula sua própria média

Em vez de calcular a média no main, a **própria classe** faz isso:

```
public double calcularMedia() {  
    return (nota1 + nota2) / 2;  
}  
  
public boolean estaAprovado() {  
    return calcularMedia() >= 7;  
}
```

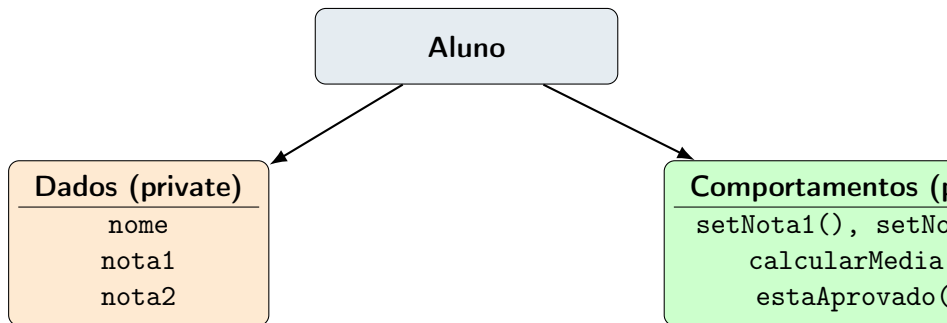
No main, o código fica simples e limpo:

```
Aluno aluno = new Aluno("Joao", 8.5, 7.0);  
  
System.out.println("Media: " + aluno.calcularMedia()); // 7.75  
if (aluno.estaAprovado()) {  
    System.out.println("Aprovado!");  
}
```

Boa prática

Coloque dentro da classe os **comportamentos** que pertencem ao **objeto**. Não jogue responsabilidades no main!

Aluno: dados + comportamentos juntos



Princípio

Um objeto deve reunir seus **dados** e seus **comportamentos**. Isso é a essência da orientação a objetos.

Comparando: sem vs. com encapsulamento

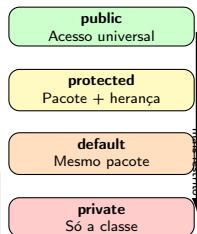
✗ Sem encapsulamento	✓ Com encapsulamento
Atributos públicos	Atributos privados
Qualquer código altera	A classe controla alterações
Valores inválidos aceitos	Valores são validados
Regras espalhadas	Regras centralizadas na classe
Difícil de depurar	Fácil de encontrar erros
Mais fácil gerar inconsistência	Objeto sempre em estado válido
Código desorganizado	Código bem organizado

Resumo dos modificadores de acesso

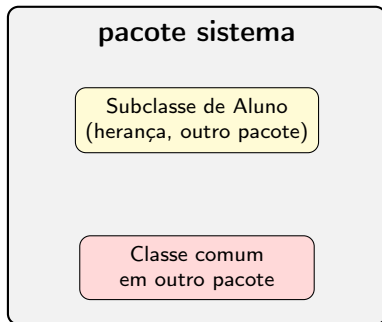
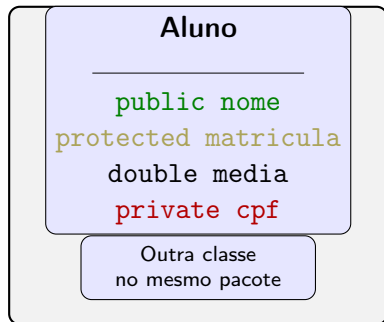
Modificador	Quem pode acessar?
<code>public</code>	Todos — qualquer classe, qualquer pacote
<code>protected</code>	Mesmo pacote + subclasses
<code>default</code>	Apenas o mesmo pacote
<code>private</code>	Apenas a própria classe

💡 Regra de ouro

Quanto **mais restrito** o acesso, maior o controle sobre os dados. Para atributos: sempre prefira `private`.



Visibilidade com pacotes



public: todos acessam

protected: pacote + subclasse

default: mesmo pacote

private: só a classe

Quando usar getter?

Use getter quando...

Outro código precisa **consultar** uma informação do objeto, mas não deve alterá-la diretamente.

Bons exemplos de getter:

- `getSaldo()` — consultar saldo
- `getNome()` — consultar nome
- `getPreco()` — consultar preço
- `getMedia()` — consultar média

Cuidado

Não crie getter para tudo automaticamente. Pergunte: *"alguém realmente precisa ler esse dado?"*

Exemplo: talvez o CPF interno de um usuário **não deva ser exposto**.

Quando usar setter?

Use setter quando...

Faz sentido permitir **alteração controlada** de um atributo após a criação do objeto.

Bons exemplos de setter:

- `setPreco()` — atualizar preço (com validação)
- `setNota()` — corrigir nota (0 a 10)
- `setNome()` — corrigir nome (não vazio)
- `setIdade()` — corrigir idade (≥ 0)

✗ Não precisa de setter:

- saldo da conta bancária (use depositar/sacar)
- cpf — não deve mudar
- `dataNascimento` — não deve mudar

Exemplo: setter para nome com validação completa

```
public void setNome(String nome) {  
    if (nome != null && !nome.isBlank()) {  
        this.nome = nome;  
    }  
    // Se for invalido, simplesmente ignora (nao altera)  
}
```

Chamada	Resultado
setNome("Ana")	✓ Aceito
setNome()	✗ Ignorado (vazio)
setNome(" ")	✗ Ignorado (só espaços)
setNome(null)	✗ Ignorado (nulo)

- `nome != null` — impede valor nulo
- `!nome.isBlank()` — impede texto vazio ou só espaços

Variação: setter que informa se deu certo

Uma alternativa útil é o setter retornar boolean:

```
// Versao que retorna void (mais simples)
public void setIdade(int idade) {
    if (idade >= 0) {
        this.idade = idade;
    }
}

// Versao que retorna boolean (mais informativa)
public boolean setIdade(int idade) {
    if (idade >= 0) {
        this.idade = idade;
        return true; // Deu certo!
    }
    return false; // Valor invalido
}
```

Qual usar?

Depende da necessidade. Se o código que chama o setter precisa saber se funcionou, use o boolean. Para casos simples, void é suficiente.

Checklist: minha classe está bem encapsulada?

✓ Deve ser SIM

- ☐ Atributos estão `private`?
- ☐ Getters existem onde necessário?
- ☐ Setters validam valores?
- ☐ Regras estão dentro da classe?
- ☐ A classe cuida dos seus dados?

✗ Deve ser NÃO

- ☐ Tem atributo `public`?
- ☐ Tem setter sem validação?
- ☐ Tem setter desnecessário?
- ☐ Regras de negócio no `main`?
- ☐ Valores inválidos passam?

💡 Dica

Antes de entregar qualquer classe, passe por esse checklist!

Exercício 1: Conta bancária

Crie uma classe ContaBancaria com:

- Atributos privados: String titular e double saldo
- Construtor recebendo o titular (saldo começa em 0)
- depositar(double valor) — rejeita valores ≤ 0
- sacar(double valor) — rejeita se saldo insuficiente
- getSaldo() e getTitular()

Teste no Main

```
ContaBancaria c = new ContaBancaria("Ana");  
c.depositar(500);  
System.out.println(c.sacar(200)); // true  
System.out.println(c.sacar(400)); // false (saldo insuficiente)  
System.out.println(c.getSaldo()); // 300.0
```


Exercício 2: Corrigir a classe Produto

Corrija a classe abaixo aplicando encapsulamento:

```
public class Produto {  
    String nome;  
    double preco;  
    int quantidade;  
}
```

Implemente:

- Atributos private + construtor com validação
- setPreco() — não aceita valor negativo
- setQuantidade() — não aceita valor negativo
- adicionarEstoque(int qtd) — soma ao estoque
- removerEstoque(int qtd) — remove se houver quantidade suficiente
- calcularValorTotal() — retorna preco * quantidade

Exercício 3: Aluno

Crie a classe Aluno com:

- Atributos privados: String nome, double nota1, double nota2
- setNota1() e setNota2() — apenas aceita entre 0 e 10
- calcularMedia() — retorna a média das duas notas
- estaAprovado() — retorna true se média ≥ 7

Teste no Main

```
Aluno a = new Aluno("Carlos", 8.0, 6.5);  
System.out.println(a.calcularMedia()); // 7.25  
System.out.println(a.estaAprovado()); // true  
a.setNota1(15); // Deve ser ignorado (fora do intervalo)  
System.out.println(a.calcularMedia()); // Ainda 7.25
```

Exercício 4: Identifique e corrija o erro

O código abaixo está correto em termos de encapsulamento?

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public void setIdade(int idade) {  
        this.idade = idade; // Sem validacao!  
    }  
}
```



Pergunta

O que acontece se alguém chamar setIdade(-20)?

Sua tarefa: Reescreva setIdade para:

- Impedir idades negativas
- Impedir idades acima de 150 (valor absurdo)
- Retornar boolean indicando se a operação foi bem-sucedida

Solução do Exercício 4

```
// Versao simples (apenas idade >= 0)
public void setIdade(int idade) {
    if (idade >= 0) {
        this.idade = idade;
    }
}
```

```
// Versao completa (com limite maximo e retorno boolean)
public boolean setIdade(int idade) {
    if (idade >= 0 && idade <= 150) {
        this.idade = idade;
        return true;
    }
    return false; // Idade invalida
}
```

Reflexão

A segunda versão é mais robusta e permite que o código que chama o método saiba se a operação foi bem-sucedida. Prefira quando a validação for importante para o fluxo do programa.

Miniatividade final: Cofrinho

Modele a classe Cofrinho:

Atributos privados:

- String dono
- double valorGuardado

Métodos:

- adicionarMoeda(double valor)
- quebrarCofrinho()
- getValorGuardado()
- getDono()

</> Teste esperado

```
1 Cofrinho c = new Cofrinho("Pedro");
2 c.adicionarMoeda(1.50);
3 c.adicionarMoeda(0.50);
4 c.adicionarMoeda(-2.0); // Ignorar!
5 System.out.println(
6     c.getValorGuardado()); // 2.0
7 double resgate = c.quebrarCofrinho();
8 System.out.println(resgate); // 2.0
9 System.out.println(
10    c.getValorGuardado()); // 0.0
```

⚠ Regras

Valor negativo deve ser ignorado. Ao quebrar, o cofre zera e retorna o valor acumulado.



Conceitos-chave

- Encapsulamento protege os dados
- Atributos → `private`
- Métodos públicos controlam acesso
- Getters para consultar
- Setters com validação para alterar



Lembrar sempre

- Nem todo atributo precisa de setter
- Setter sem validação é inútil
- A classe deve centralizar suas regras
- Objeto deve estar sempre em estado válido

“O objeto deve ser responsável por seus próprios dados.”